

Solving copyright & licensing information in client-side JavaScript, CSS etc.

REUSE.software for JS, CSS &sim – how it all comes together

Matija Šuklje

2026-09-14

Open Regulatory Compliance Work Group (ORC WG)

what

what

... is the problem?

problem of client-side JS, CSS etc.

JS & CSS (&sim.) client-side code

- no notices at all, nor licenses
- even if in source, no notices when distributed

= copyright problem

what

... are the goals

make copyright & license data useful for compliance

in a manner that is

- short
- simple
- no new specs – rely on existing standards, *as far as possible*
- no new tooling – rely on common tools, *as far as possible*

solution

solution

tl;dr

how to equip browser-side JS/CSS/... with compliance metadata

1. apply **REUSE.software** best practices
2. use a **tool of *your choice* to generate:**
 - an SBOM – [SPDX](#), [CycloneDX](#), or whatever you prefer; and/or
 - an attribution file, of whatever kind you prefer; and/or;
 - whatever document/file/webpage you feel is the most appropriate way to fulfill your licensing obligations
3. make sure that **the file/page from 2. is linked in a `<link rel="license" href="">` HTML tag** in the HTML `<head>`
4. create something **in the web UI to present this link too, using a `<a rel="license" href="">` HTML tag** in the HTML `<body>`

why does it work

why does it work

REUSE

use REUSE to add metadata

in every source code file¹ – clarity; metadata follows the code:

```
/*  
 * SPDX-CopyrightText: © 2026 Hacke R. McRandom <hacker@mcrandom.example>  
 * SPDX-LicenseIdentifier: MIT OR EUPL-1.2  
 */
```

store license texts in `LICENSES/$spdx_id.txt` – clarity; predictability; quick overview

¹If you wonder about the structure of the copyright notice, I have a long blog about [How and why to properly write copyright statements in your code](#).

why does it work

SBOMs and docs

different organisations and projects will have different needs and risks

suggestions:

- an **SBOM** – machine-readable
- a human-readable **document** (e.g. “attribution” HTML)

have a tool generate both (e.g. ScanCode, FOSSology, ORT, CaPyCLI, ...), ideally automated

why does it work

distribute the info

use a `<link>` or `<a>` tag, respectively, with `rel=license` attribute and the appropriate MIME type.

`rel=license`:

*Valid on the `<a>`, `<area>`, `<form>`, `<link>` elements, the license value indicates that the **hyperlink leads to a document describing the licensing information**; that the main content of the current document is covered by the copyright license described by the referenced document. If not inside the `<head>` element, the standard doesn't distinguish between a hyperlink applying to a specific part of the document or to the document as a whole. Only the data on the page can indicate this.*

```
<head>
  <link
    rel="license"
    href="sbom.spdx.json"
    type="application/spdx+json"
    fetchpriority="high"
  />
</head>
```

distribute it to humans

```
<body>
[...]
```

`<footer>`

This website is licensed under

`<a rel="license" href="https://creativecommons.org/licenses/by/4.0/">CC-BY-SA-4.0</a>`

with the client-side software and styling under several FOSS licenses - the information to which, you can get in

`<a rel="license" href="attribution.html" type="text/html">human-readable</a>` and

`<a rel="license" href="sbom.spdx.json" type="application/spdx+json">machine-readable</a>` form.

`</footer>`

```
</body>
```

impact

impact

performance vs risk

what is the risk trade-off

it may be important (e.g. in Germany) that license is shared at the *exact* same time as the content it covers

unless I heavily misunderstand how HTML (and HTTP) work:

- the **minified JS/CSS** itself will *not increase* a bit;
- **you can tweak the “speed”** of your webpage loading by setting the `fetchpriority` attribute to either
 - low, if you prefer the page to *load faster* but and OK with the risk²; or
 - high if you are *risk averse* and OK with the page loading slower.

²If you feel this is not fast enough either, you could even try out the `rel=preload` attribute in combination with `<link as="">`. But, to be honest, I would expect some big ire from the web developers if you did that, and there seems to be no ideal `as=""` to “preload as” either, because `license` is not an option there.

impact

garbage in = garbage out

let's fix GI=GA!

this does not itself solve the original *garbage in* problem – someone needs to add the tags

but it provides a solution on ***how* to easily, without much extra work and tooling:**

- **store this data** – just use [REUSE.software](#) as in every other programming language; and
- **provide this data** – serve the results in `<link rel="license">` and `<a rel="license">`

relevant for EU CRA?

relevant for EU CRA?

maybe? depends

Example 6: “a *website that merely presents information to its visitors* and does not support the functionality of a product with digital elements is not itself a product with digital elements.”:

- out of scope

Example 6: “a *website that merely presents information to its visitors* and does not support the functionality of a product with digital elements is not itself a product with digital elements.”:

- out of scope

Example 4: “a *desktop application built using web technologies* but packaged for *local installation* is supplied to the user and executes on the user’s device, and is therefore a product with digital elements.”:

- **CRA likely applies**

Example 5: “a web application accessed by the user exclusively through a web browser is not a product with digital elements. *Unless it supports the functionality of a product with digital elements*, it does not fall within the scope of the CRA. By contrast, an application supplied to the user as a *locally installed client that executes on the user’s device* is a product with digital elements. Where it is placed on the market in the course of a commercial activity, it may fall within the scope of the CRA. If that client relies, in order to perform one or more of its functions, on data processing at a distance which meets the definition of remote data processing, that data processing at a distance is also part of the product with digital elements.”:

- CRA **may apply**, depending on factors

discussion

discussion

q & a

fin

fin

thank you

work: matija.suklje@liferay.com

personal: matija@suklje.email

more: <https://matija.suklje.name/contact>

bonus: first (controversial) proposal

bonus: first (controversial) proposal

goals

just **how far** *can* we get with existing specs and tools? >;)=

- short
- simple
- no new specs – rely on existing standards
- no new tooling – rely on common tools

bonus: first (controversial) proposal

actual suggestion

actual suggestion: how it goes

- put copyright and license info into source code
- know what's what even when mixed with unmarked code
- (hopefully) have the info stay in place *even when minified*

[blog post of the original suggestion](#)

[FOSDEM workshop on this topic](#)

actual suggestion: how it looks

```
/*!  
 * SPDX-SnippetBegin  
 * SPDX-SnippetCopyrightText: © 2024 Hacke R. McRandom <hacker@mcrandom.example>  
 * SPDX-LicenseIdentifier: MIT OR Unlicense  
 */  
  
import half_of_npm  
  
code_goes_here();  
  
/*! SPDX-SnippetEnd */
```

bonus: first (controversial) proposal

did it work?

bonus: first (controversial) proposal

in theory

support in minifiers

minifier	@license (JSDoc)	/*! (YUI)
Google Closure Compiler	yes	yes ³
Esbuild	yes	yes
JSrink	yes	yes
SWC ⁴	yes	yes
Terser	yes	yes
UglifyJS	yes	yes
YUI Compressor	no	yes
Bun	no	no (but, wip)

³It is not documented but [Google Closure Compiler](#) does indeed support ! and treats it explicitly as a license block (without any additional JSDoc parsing) [since 2016](#).

⁴SWC is primarily a compiler and module bundler, but I keep it here because it does its own minification instead of relying on an external tool.

bonus: first (controversial) proposal

in practice

Svelte + Rollup + Terser

result: *mostly* fine

CSS:

- in `bundle.css` no SPDX tags, but OK since it was all 3rd party :)?

JS:

- in `bundle.js` all SPDX tags remain :)
- but one `SnippetEnd` tag came up before the `SnippetBegin` tag :(
 - “problem” in how Terser associates comments to functions, fixable
- *but* in `bundle.js.map` tags are perfectly fine :)

PoC 2: React scripts

React Scripts (+ WebPack + Terser)

result: *mostly* fine – React = opinionated, v.hard to change

CSS:

- tags kept in both `main.05c219f8.css` and `main.05c219f8.css.map` :)

JS:

- `main.608edf8e.js` no SPDX tags (or any other comments) :(
- but `main.608edf8e.js.LICENSE.txt` has all the SPDX-* tags as well other important comments (e.g. @license blocks from 3rd party components) :)?
 - *all* license-relevant comments dumped into `main.608edf8e.js.LICENSE.txt` (useless)
- and `main.608edf8e.js.map` includes SPDX-* tags, as expected :)

Rollup + WebPack + Terser

we⁵ planted the SPDX tags in every *.js file during build

result: *very fine*

JS:

- in `rollup.browser.js` all SDPX tags seem to be in order; there is just some un-tagged code at the very bottom :)
- no tags in `rollup.browser.js.map`, but that was expected and correct

⁵It was actually Lukas Taegert-Atkinson, I merely presented the idea.

**bonus: feedback to controversial
proposal**

bonus: feedback to controversial proposal

in broad strokes

compliance ppl: “yes, makes *total* sense”

compliance ppl: “yes, makes *total* sense”

JS devs: “no way, this will make the code way too big”

**bonus: feedback to controversial
proposal**

just how BIG does it get *really*?

theory

$$V_{min.wSPDX} = V_{min} + (n_{sourcefiles} \times V_{SPDXtagblock})$$

theory

$$V_{min.wSPDX} = V_{min} + (n_{sourcefiles} \times V_{SPDXtagblock})$$

- $V_{min.wSPDX}$... size of minified package with SPDX tags preserved
- V_{min} ... size of minified package
- $n_{sourcefiles}$... number of relevant source code files
- $V_{SPDXtagblock}$... size of SPDX tag block

theory to practice

should not be too much really

many small files – bigger impact

few big files – smaller impact

added size due to SPDX tags in minified code

practice

- PoC 1 (mini Svelte app): 12.324 B vs 12.681 B ($\Delta = +3\%$)
- PoC 2 (mini React app): 536 KiB vs 536 KiB ($\Delta = +0.2\%$)
- PoC 3 (Rollup Browser⁶): 374 KiB vs 415 KiB ($\Delta = +11\%$)

⁶NB: Rollup Browser PoC was the only one where even 3rd party files had added SPDX tags. In the others it was just 1st party code that we added SPDX tags to.

bonus: feedback to controversial proposal

more specific feedback

use `REUSE.toml` instead and [banner with license info](#)

problem with this counter-proposal:

- solution depends on specific tool (Rollup)
- info is missing in source code – will get lost

big projects challenges from a big company

the proposal sounds like the proper way to go

but big project challenges:

- WebPack, UglifyJS etc. may over-preserve comments that are tagged as important, but aren't really (e.g. whole README)
 - performance hit; even worse with WebAssembly
 - problem is really in the misuse (or different interpretation) of what is “important” – potential argument to use `@license` instead of `/*!`

big projects challenges from a big company

the proposal sounds like the proper way to go

but big project challenges:

- WebPack, UglifyJS etc. may over-preserve comments that are tagged as important, but aren't really (e.g. whole README)
 - performance hit; even worse with WebAssembly
 - problem is really in the misuse (or different interpretation) of what is “important” – potential argument to use `@license` instead of `/*!`

current workaround applied by this company:

- create an SBOM relying on package manager data (and pretend data is good enough) and ship SBOM along minified JS